

Enter NAME and CID here

Question 1 (75 marks)

The only non-base-R packages you are allowed to load in this question are `igraph`, `R6` and `testthat`.

Your task is to build an R package that implements and analyses discrete-time Markov chains using R6 classes.

Background

Let $(X_t)_{t \in \mathbb{N}}$ be a *discrete-time time-homogeneous Markov chain* with finite state space $S = \{1, \dots, K\}$ for $K \in \mathbb{N}$. Let $P \in \mathbb{R}^{K \times K}$ be the transition matrix of the Markov chain which contains the transition probabilities $P_{ij} = \Pr(X_{t+1} = j | X_t = i)$. We mention here the following facts from the theory of Markov chains:

- (1) If $\pi \in \mathbb{R}^K$ is a vector describing the initial distribution of the chain over the state space S (so the components of π are between 0 and 1 and sum to 1), then the marginal distribution π_t of X_t over S for $t \in \mathbb{N}$ can be obtained from $\pi_t = \pi P^t$, where $P^t = PP \cdots P$ is the t -fold matrix product of P .
- (2) A distribution π is called *stationary* if $\pi = \pi P$.
- (3) If there exists a unique stationary distribution π (i.e. if the chain is irreducible since we are on a finite state space), then the *power method* says that for any initial distribution π_0 we have convergence of $\pi_0 P^t$ to π as t tends to infinity.
- (4) The *ergodic theorem* for Markov chains says that if there exists a unique stationary distribution π , then for any initial state $X_0 = i \in \{1, \dots, K\}$ we have $N^{-1} \sum_{t=1}^N \mathbf{1}(X_t = i) \rightarrow \pi(i)$ as N tends to infinity where $\mathbf{1}(X_t = i)$ equals 1 when $X_t = i$ and zero otherwise, and where $\pi(i)$ is the π -probability of being in state i . This means that the relative frequency of being in state i converges along any path to $\pi(i)$.

Code clarity (5 marks)

There is a famous saying among software developers that code is read more often than it is written. Marks will be awarded (or not awarded) based on the clarity of the code and appropriate use of comments.

Part A (18 marks)

Create a file `markovChain.R` containing an R6 class `MarkovChain` with at least the following fields and methods:

- (1) fields:
 - `initial_state` - an index from $1, \dots, K$ with the initial state
 - `P` - a quadratic transition matrix of dimension $K \times K$ (entries must be positive, between 0 and 1 and the row sums should be 1)
- (2) methods:
 - `initialize` - takes as inputs a transition matrix `P` and an `initial_state` and sets the fields in (1)

- **simulate** - takes as inputs a number of steps `n_steps` and a number of distinct simulation paths `n_paths`, simulates `n_paths` independent trajectories of length `n_steps` all starting from `initial_state`, returns a convenient R object with the simulation results
- **marginal_distribution** - takes as input a time `t`, returns the marginal distribution at time `t` when starting from the initial distribution with point mass at `initial_state`
- **empirical_marginal_distribution** - takes as inputs paths of a Markov chain simulation (obtained from **simulate**) and returns the empirical distribution at each of the time points (i.e. for each state at this time point the relative frequency along the paths)
- **stationary_distribution** - takes as inputs a tolerance value `tol` and a maximum number of iterations `max_iter` and returns an estimate of the stationary distribution (assuming there exists one) by the power method starting from the initial distribution with point mass at `initial_state`
- **visualise** - takes no inputs and prints the directed weighted graph corresponding to the transition matrix using `igraph::graph_from_adjacency_matrix()` from the **igraph** package and plots it; edge labels reflect transition probabilities.

Observe the following:

- Include appropriate (but brief) documentation for all methods.
- Use defensive programming techniques to check for incorrect input.
- Fields and methods may be public or private.
- You may add additional fields and methods if you think this is helpful.
- Part B gives marks for appropriate unit tests. Try to follow a test based development approach when implementing the class and methods.

Part B (10 marks)

Create a file `test-markovChain.R` with appropriate unit tests for the class and methods from part A except for the **stationary_distribution** and **visualise** methods. Write at least one test per method and one test for the class itself.

Part C (12 marks)

Create a folder `markovSim` using `devtools` that follows the standard R package structure. Add the R source files from parts A and B in the appropriate places. Take care of the following points:

- All unit tests from part B should pass.
- Make appropriate modifications of the DESCRIPTION file. Keep the details of the package brief.

Create an R package called `markovSim`. The result should be a file named `markovSim_0.0.1.tar.gz` if you have chosen version number 0.0.1.

Part D (15 marks)

Install and load the package `markovSim`. Use the code blocks below to answer the following questions.

Part D(i) (8 marks)

Consider the Markov chain starting from state 1 and with transition matrix

$$P = \begin{pmatrix} 0.6 & 0.4 \\ 0.3 & 0.7 \end{pmatrix}$$

Instantiate a `MarkovChain` object from the initial state and transition matrix `P`. Perform the following operations and print out the results:

- Visualise the transition matrix using the **visualise** method.

provide your answer here

(ii) Simulate 20 paths of length 10.

```
# set seed to ensure results are reproducible  
set.seed(42)
```

provide your answer here

(iii) Compute the marginal distribution after 5 steps.

provide your answer here

(iv) Compute the empirical marginal distributions along the 10 simulated paths.

provide your answer here

(v) Compute an approximate stationary distribution.

provide your answer here

(vi) Demonstrate the result stated in item (4) of the background section (convergence of relative frequencies for all states).

provide your answer here

Part D(ii) (7 marks)

Consider the transition matrix P defined in the following R snippet

```
P <- matrix(c(  
  0.40, 0.30, 0.20, 0.10, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00,  
  0.10, 0.50, 0.20, 0.20, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00,  
  0.20, 0.20, 0.40, 0.20, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00,  
  0.30, 0.10, 0.20, 0.40, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00,  
  0.00, 0.00, 0.00, 0.00, 0.30, 0.20, 0.30, 0.20, 0.00, 0.00, 0.00, 0.00,  
  0.00, 0.00, 0.00, 0.00, 0.20, 0.40, 0.20, 0.20, 0.00, 0.00, 0.00, 0.00,  
  0.00, 0.00, 0.00, 0.00, 0.10, 0.30, 0.40, 0.20, 0.00, 0.00, 0.00, 0.00,  
  0.00, 0.00, 0.00, 0.00, 0.20, 0.10, 0.30, 0.40, 0.00, 0.00, 0.00, 0.00,  
  0.20, 0.10, 0.10, 0.10, 0.10, 0.10, 0.00, 0.00, 0.15, 0.15, 0.00, 0.00,  
  0.10, 0.10, 0.00, 0.00, 0.10, 0.10, 0.10, 0.00, 0.00, 0.20, 0.15, 0.15,  
  0.05, 0.05, 0.05, 0.05, 0.10, 0.10, 0.10, 0.10, 0.10, 0.00, 0.20, 0.10,  
  0.00, 0.00, 0.10, 0.10, 0.05, 0.05, 0.05, 0.05, 0.20, 0.20, 0.10, 0.10  
) , nrow = 12, byrow = TRUE)
```

Use the class and methods from part A to show that a Markov chain with this transition matrix cannot have a unique stationary distribution.

provide your answer here

Part E (15 marks)

Considering all the topics we have covered in the module, give an example of something that, as a language, R does better than Python. It could also be a feature which R has, but Python does not. There is more than one correct answer, but you may provide *only one answer*. You may *not* use examples which refer to topics in future modules such as “R is better at regression” or “Python is better at deep learning”. Refer to topics or features you have covered in this module, but do not refer to statistical or machine learning procedures covered in future modules.

Besides describing the example, you also need to provide accompanying scripts in both R and Python that demonstrate the advantage described by your example. A significant proportion of the marks is allocated to providing the supporting example scripts and screenshots.

For your solution of this part of the question, create a folder **Q1E** containing the following:

- A text file **q1e.txt** where you describe your answer.
- Example scripts **q1e.R** and **q1e.py** in R and Python, and any data files, to support your answer.
- (optional) Screenshots in **.png**, **.jpg** or **.pdf** format if appropriate to your answer.